

Project Report

POTATO GPU

rvlab14:

Ruslan Kharasov, 0486519, Technische Informatik Bachelor (kharasov@campus.tu-berlin.de)

Lukas Schwarzmeier, 393158, CE Master (l.schwarzmeier@campus.tu-berlin.de)

David Schröder (Schülerstudent) (david.schroeder.2@campus.tu-berlin.de)

Silvan Josten, 410678, Elektrotechnik Master (silvan.josten@campus.tu-berlin.de)

July 29, 2026

Contents

1	contributions	3
2	Requirements and functional specification	3
2.1	Requirements	3
2.2	Functional Specification	3
2.3	Stretch Goals	3
3	High level design	4
4	Results	4
4.1	Specifications and Stretchgoals	4
4.2	Remarks	4
4.3	Photos	6
4.4	Links	6

1 contributions

	David	Silvan	Ruslan	Lukas
rasterizer			100%	
vertex processor		50%		50%
hdmi controller	100%			
main application loop	100%			
graphics driver	100%			
pipeline integration				100%

2 Requirements and functional specification

2.1 Requirements

Aim of the project is to implement a high-resolution video output controller for 3D graphics in an interactive environment.

2.2 Functional Specification

- Video Output
 - HDMI
 - 60 Hz refresh rate
 - 24-bit color
 - 1920x1080p resolution
- Input management: Joystick input
- Software Rendering
 - Mesh Rasterization approach
 - Scene consisting of single model
 - 3D projection logic
 - Camera logic with interactive translation + rotation
 - UV mapped textures
 - Depth buffer

2.3 Stretch Goals

- Hardware-accelerated matrix multiplication (included in HW diagram)
- Hardware-accelerated rasterization using multi-core rasterizer topology
- Support for multiple models in the scene via model matrices

3 High level design

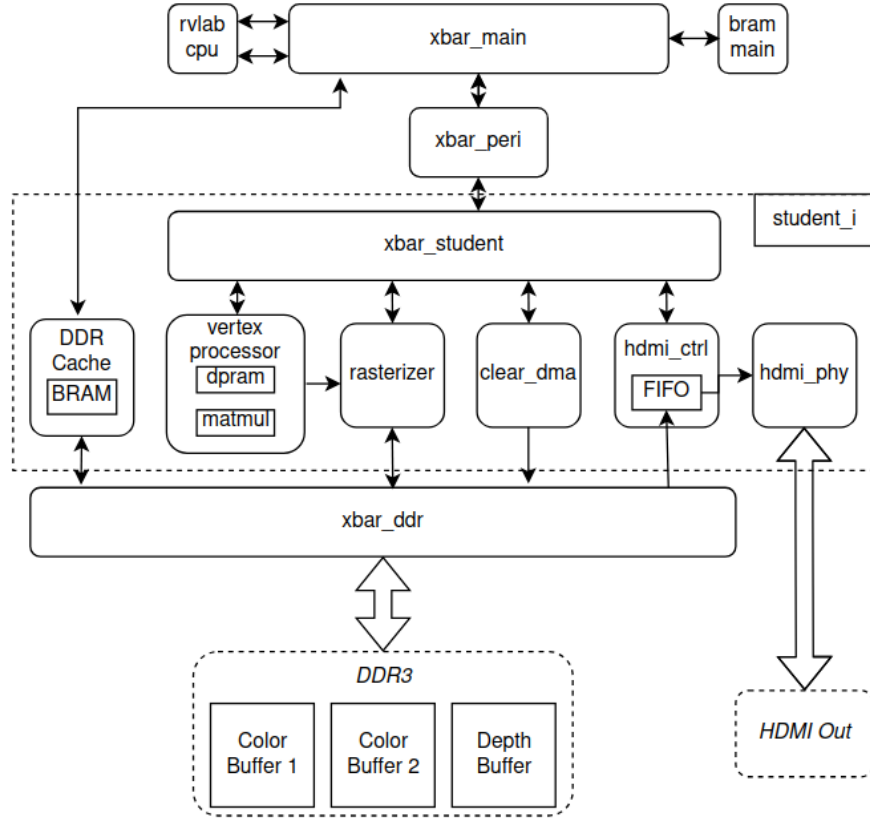


Figure 1: Hardware Top-Level Diagram

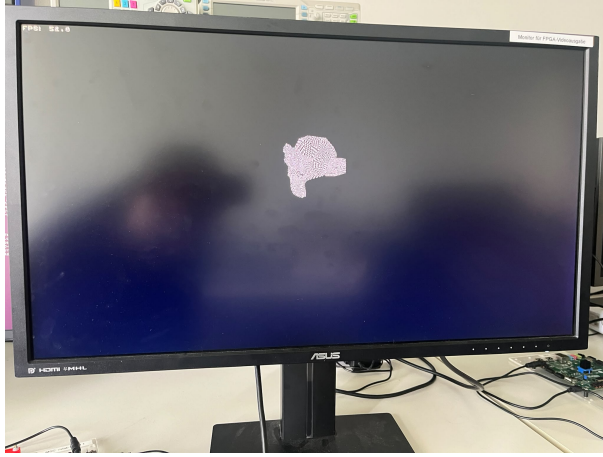
4 Results

4.1 Specifications and Stretchgoals

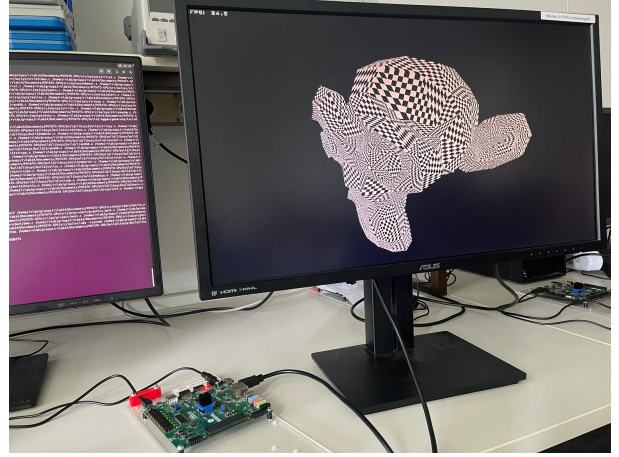
- all but one of the functional specification parameters were met
- due to it's unavailability the joystick input was replaced by a keyboard fully satisfying the interactivity requirement
- Hardware-accelerated matrix multiplication was implemented
- Hardware-accelerated rasterization using multi-core rasterizer topology was implemented

4.2 Remarks

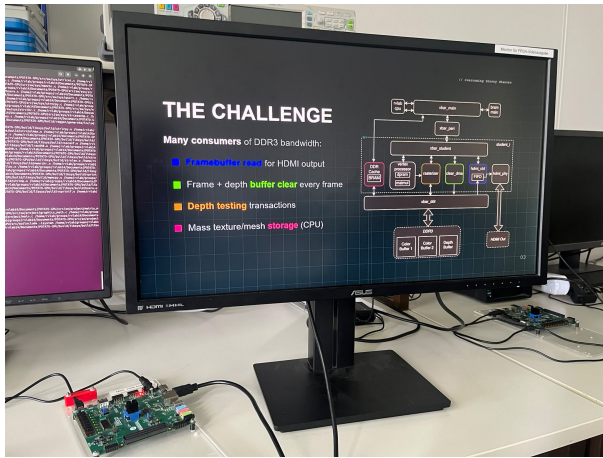
- while the 60Hz refresh rate was met, the rendered frames per second fall with a growing number of colored pixels on the screen
- the rasterizer computes around 2.9 pixels per cycle
- as the rams' bandwidth is fully utilized further optimization would yield deminsing results,



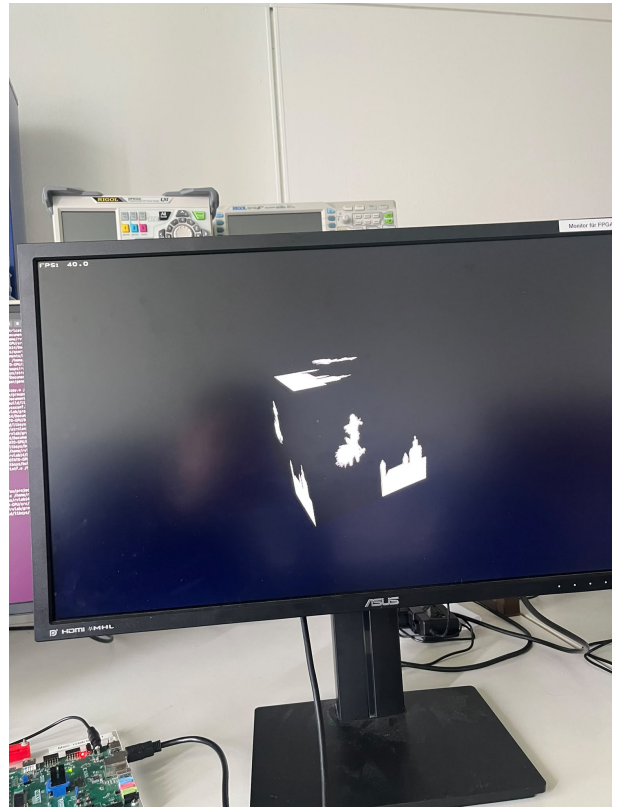
(a)



(b)



(c)



(d)

Figure 2: Results of the FPGA-based graphics implementation. As depicted in the images a and b the frame rate depends on the number of pixels shaded. This explains why the frame rate is 52 FPS in image a, compared with 24.5 FPS in image b.

4.3 Photos

4.4 Links

- Presentation Slides and video: <https://tubcloud.tu-berlin.de/s/YwS49454FnFtaPX>